

Guía Completa: Subir Proyectos a GitHub

Una guía paso a paso para subir cualquier proyecto a GitHub desde tu máquina local.

Índice

1. [Prerrequisitos](#)
2. [Configuración Inicial](#)
3. [Crear Repositorio en GitHub](#)
4. [Preparar el Proyecto Local](#)
5. [Subir Código a GitHub](#)
6. [Comandos Git Básicos](#)
7. [Flujo de Trabajo Diario](#)
8. [Solución de Problemas](#)
9. [Buenas Prácticas](#)

Prerrequisitos

1. Instalar Git

- **Windows:** Descargar desde git-scm.com
- **macOS:** `brew install git` (con Homebrew) o descargar desde git-scm.com
- **Linux:** `sudo apt-get install git` (Ubuntu/Debian) o `sudo yum install git` (CentOS/RHEL)

2. Crear Cuenta en GitHub

- Ir a github.com
- Registrarse con email y contraseña
- Verificar email

3. Verificar Instalación

```
git --version
```

Configuración Inicial

Configurar Git (solo una vez)

```
# Configurar nombre de usuario
git config --global user.name "Tu Nombre Completo"

# Configurar email (usar el mismo de GitHub)
git config --global user.email "tu-email@ejemplo.com"
```

```
# Configurar editor por defecto (opcional)
git config --global core.editor "code" # Para VS Code
git config --global core.editor "notepad" # Para Notepad (Windows)

# Verificar configuración
git config --list
```

Configurar Autenticación

Opción 1: Personal Access Token (Recomendado)

1. En GitHub: Settings → Developer settings → Personal access tokens → Tokens (classic)
2. Generate new token → Generate new token (classic)
3. Seleccionar scopes: `repo`, `workflow`
4. Copiar el token (se muestra solo una vez)

Opción 2: SSH Keys

```
# Generar clave SSH
ssh-keygen -t ed25519 -C "tu-email@ejemplo.com"

# Ver clave pública
cat ~/.ssh/id_ed25519.pub

# Copiar la clave a GitHub: Settings → SSH and GPG keys → New SSH key
```

Crear Repositorio en GitHub

1. Crear Nuevo Repositorio

1. Ir a github.com
2. Clic en **"New"** o **"+"** → **"New repository"**
3. Completar información:
 - **Repository name:** `nombre-del-proyecto`
 - **Description:** Descripción breve del proyecto
 - **Visibility:** Public (público) o Private (privado)
 - **NO** marcar "Add a README file" (si ya tienes uno)
 - **NO** marcar ".gitignore" (si ya tienes uno)
 - **NO** marcar "Choose a license" (si ya tienes uno)

2. Copiar URL del Repositorio

- **HTTPS:** `https://github.com/TU_USUARIO/nombre-del-proyecto.git`
 - **SSH:** `git@github.com:TU_USUARIO/nombre-del-proyecto.git`
-

Preparar el Proyecto Local

1. Crear .gitignore

Crear archivo `.gitignore` en la raíz del proyecto:

```
# Para proyectos Node.js
node_modules/
npm-debug.log*
yarn-debug.log*
yarn-error.log*
.env
.env.local
.env.development.local
.env.test.local
.env.production.local

# Para proyectos React/Vite
dist/
build/

# Archivos del sistema
.DS_Store
Thumbs.db

# IDEs
.vscode/
.idea/
*.swp
*.swo

# Logs
*.log
logs/

# Archivos temporales
tmp/
temp/
```

2. Crear README.md

```
# Nombre del Proyecto

Descripción breve del proyecto.

## Instalación

```bash
npm install
```

## Uso

```
npm start
```

## Tecnologías

- Tecnología 1
- Tecnología 2

## Licencia

MIT

```
3. Crear LICENSE (opcional)
- MIT License (más común)
- Apache 2.0
- GPL v3

📁 Subir Código a GitHub

Método 1: Proyecto Nuevo (Recomendado)

```bash
# 1. Navegar al directorio del proyecto
cd ruta/a/tu/proyecto

# 2. Inicializar repositorio Git
git init

# 3. Añadir todos los archivos
git add .

# 4. Hacer el primer commit
git commit -m "Initial commit: Descripción del proyecto"

# 5. Añadir repositorio remoto
git remote add origin https://github.com/TU_USUARIO/nombre-del-proyecto.git

# 6. Cambiar nombre de rama principal (opcional, pero recomendado)
git branch -M main

# 7. Subir al repositorio
git push -u origin main
```

Método 2: Proyecto Existente

```
# Si ya tienes un repositorio Git local
git remote add origin https://github.com/TU_USUARIO/nombre-del-proyecto.git
git branch -M main
git push -u origin main
```

Método 3: Clonar y Modificar

```
# 1. Clonar repositorio existente
git clone https://github.com/TU_USUARIO/nombre-del-proyecto.git
cd nombre-del-proyecto

# 2. Hacer cambios en el código

# 3. Añadir y commitear cambios
git add .
git commit -m "Descripción de los cambios"

# 4. Subir cambios
git push
```

Comandos Git Básicos

Estado y Información

```
# Ver estado del repositorio
git status

# Ver historial de commits
git log

# Ver historial simplificado
git log --oneline

# Ver diferencias
git diff
```

Trabajar con Archivos

```
# Añadir archivo específico
git add archivo.js

# Añadir todos los archivos
git add .
```

```
# Eliminar archivo del staging
git reset archivo.js

# Ver archivos en staging
git diff --cached
```

Commits

```
# Hacer commit con mensaje
git commit -m "Descripción del cambio"

# Hacer commit de todos los cambios (sin add)
git commit -am "Descripción del cambio"

# Modificar último commit
git commit --amend -m "Nuevo mensaje"
```

Ramas

```
# Ver ramas
git branch

# Crear nueva rama
git branch nombre-rama

# Cambiar de rama
git checkout nombre-rama

# Crear y cambiar a nueva rama
git checkout -b nombre-rama

# Eliminar rama
git branch -d nombre-rama
```

Sincronización

```
# Descargar cambios del remoto
git pull

# Subir cambios al remoto
git push

# Ver repositorios remotos
git remote -v
```

```
# Añadir repositorio remoto
git remote add origin URL
```

Flujo de Trabajo Diario

1. Antes de Empezar

```
# Actualizar repositorio local
git pull origin main
```

2. Durante el Desarrollo

```
# Ver cambios
git status

# Añadir cambios
git add .

# Hacer commit
git commit -m "Descripción clara del cambio"
```

3. Al Finalizar

```
# Subir cambios
git push origin main
```

4. Trabajar con Ramas (Recomendado)

```
# Crear rama para nueva funcionalidad
git checkout -b feature/nueva-funcionalidad

# Hacer cambios y commits
git add .
git commit -m "Añadir nueva funcionalidad"

# Subir rama
git push origin feature/nueva-funcionalidad

# Volver a main y hacer merge
git checkout main
git pull origin main
git merge feature/nueva-funcionalidad
git push origin main
```

Solución de Problemas

Error: "fatal: remote origin already exists"

```
# Ver repositorios remotos
git remote -v

# Eliminar origin existente
git remote remove origin

# Añadir nuevo origin
git remote add origin https://github.com/TU_USUARIO/nombre-del-proyecto.git
```

Error: "fatal: refusing to merge unrelated histories"

```
# Forzar merge de historiales no relacionados
git pull origin main --allow-unrelated-histories
```

Error: "fatal: The current branch has no upstream branch"

```
# Establecer upstream
git push -u origin main
```

Deshacer Último Commit

```
# Deshacer commit pero mantener cambios
git reset --soft HEAD~1

# Deshacer commit y eliminar cambios
git reset --hard HEAD~1
```

Recuperar Archivo Eliminado

```
# Ver commits donde existía el archivo
git log -- archivo.js

# Recuperar archivo de un commit específico
git checkout COMMIT_HASH -- archivo.js
```

Conflictos de Merge

```
# Ver archivos con conflictos
git status

# Resolver conflictos manualmente en el editor
# Luego:
git add .
git commit -m "Resolver conflictos de merge"
```

Buenas Prácticas

1. Mensajes de Commit

- **Formato:** `tipo: descripción breve`
- **Tipos:** `feat`, `fix`, `docs`, `style`, `refactor`, `test`, `chore`
- **Ejemplos:**

```
feat: añadir autenticación de usuarios
fix: corregir error en cálculo de densidad
docs: actualizar README con instrucciones de instalación
```

2. Estructura de Ramas

```
main (producción)
├─ develop (desarrollo)
├─ feature/nueva-funcionalidad
├─ bugfix/correccion-error
└─ hotfix/error-critico
```

3. Frecuencia de Commits

- Hacer commits pequeños y frecuentes
- Un commit por funcionalidad o corrección
- Evitar commits con múltiples cambios no relacionados

4. .gitignore

- Incluir siempre archivos específicos del proyecto
- No incluir archivos de configuración personal
- Revisar antes de hacer commit

5. README.md

- Descripción clara del proyecto
 - Instrucciones de instalación
 - Ejemplos de uso
 - Tecnologías utilizadas
 - Información de contacto
-

Comandos Rápidos de Referencia

```
# Configuración inicial
git config --global user.name "Tu Nombre"
git config --global user.email "tu-email@ejemplo.com"

# Nuevo proyecto
git init
git add .
git commit -m "Initial commit"
git remote add origin URL
git push -u origin main

# Trabajo diario
git status
git add .
git commit -m "Mensaje"
git push

# Actualizar repositorio
git pull

# Ver historial
git log --oneline
```

Recursos Adicionales

- [Documentación oficial de Git](#)
 - [GitHub Guides](#)
 - [Git Cheat Sheet](#)
 - [GitHub Desktop](#) (interfaz gráfica)
-

¡Con esta guía ya tienes todo lo necesario para subir cualquier proyecto a GitHub! 